



PROGRAMAÇÃO



Escola Naval



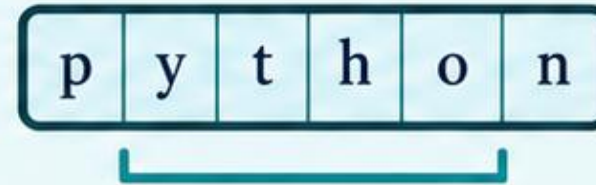
António Gonçalves

Agenda



Strings como sequências

Índices, ciclos e imutabilidade.



Extração de partes

Slicing, passos e códigos estruturados.



Pesquisa e transformação

Testes, normalização e métodos.



Processamento de texto

Split, registos e ficheiros.



Exercício final: processar dados de alunos.

Uma string é uma sequência

Uma **string** é uma sequência imutável de caracteres Unicode.

Caracteres e índices

	0	1	2	3	4	5	6
Índice:							
Carácter:	B	o	m		d	i	a

Os espaços e a pontuação também contam como caracteres na sequência.

Outros exemplos de strings

nome = 'Ana' codigo = '101' vazia = ''

Exemplo de string:

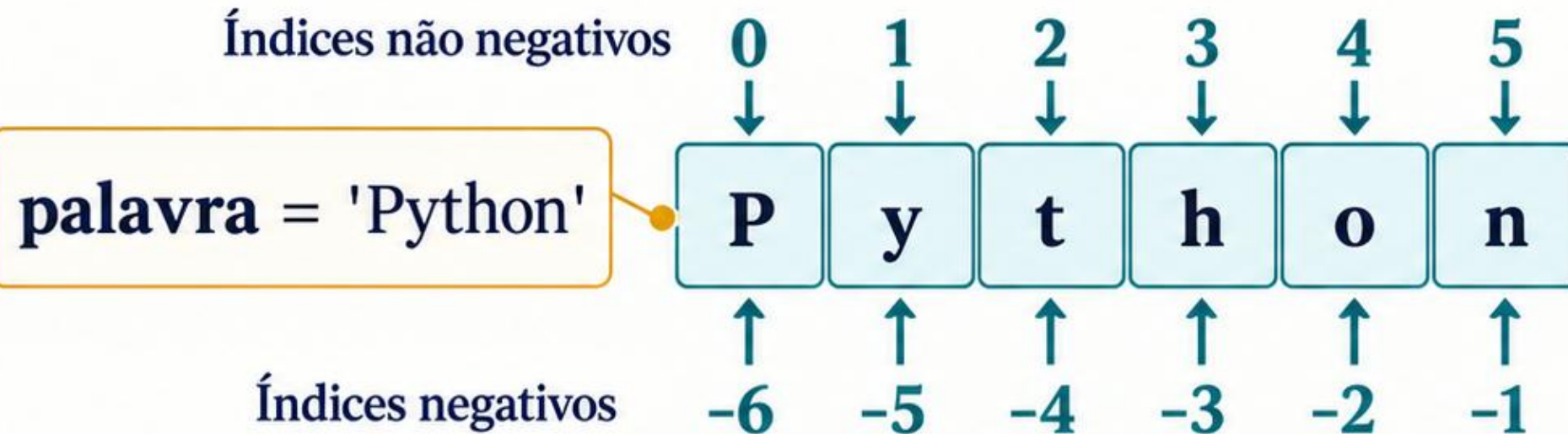
```
texto = 'Bom dia'
```

Operações com strings

Expressão	Resultado
len(texto)	7
texto[0]	'B'
texto[3]	' '
texto[-1]	'a'
type(texto)	str

O acesso a uma posição devolve uma string com um carácter.

Índices e limites



Índices não negativos válidos:
0 a `len(palavra) - 1`.

Numa *string* vazia, não existe
qualquer carácter para aceder
por índice.

Exemplos

```
>>> palavra[0]
'p'
>>> palavra[-1]
'n'
>>> palavra[-6]
'p'
>>> palavra[len(palavra) - 1]
'n'
```

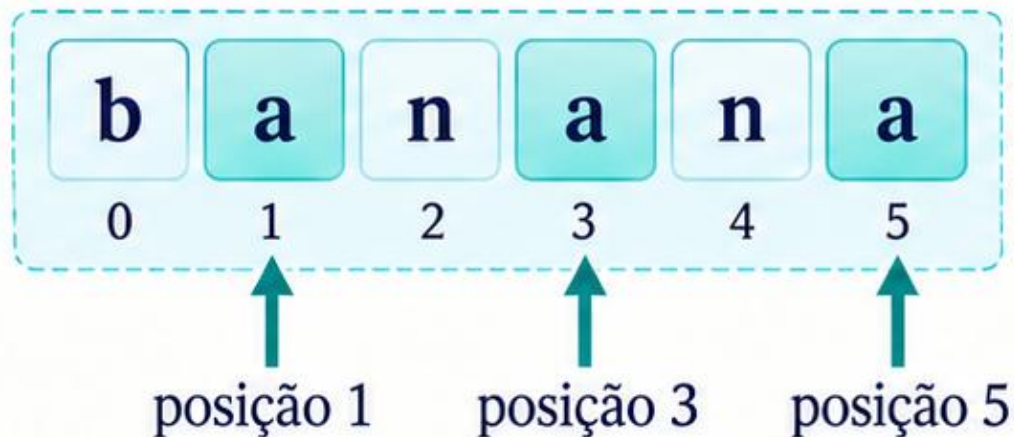
Erro

```
>>> palavra[6]
Traceback (most recent call last):
....
IndexError: string index out of range
```

Percorrer e contar caracteres

Contar: percorrer uma cadeia de caracteres e contar quantas vezes um determinado caracter aparece.

```
texto = 'banana'
quantidade = 0
for caracter in texto:
    if caracter == 'a':
        quantidade += 1
print(quantidade)
```



Saída:

3

Por índice: percorrer a cadeia de caracteres utilizando o índice para aceder a cada caracter e à sua posição.

```
texto = 'banana'
for i in range(len(texto)):
    print(i, texto[i])
```

Saída:

0	b
1	a
2	n
3	a
4	n
5	a

Por valor para ler caracteres; por índice para conhecer a posição.

Concatenação e repetição

O operador **+** junta strings.

Expressão Python	Resultado
<code>'Ana' + ' ' + 'Silva'</code>	<code>'Ana Silva'</code>
<code>'_' * 5</code>	<code>'-----'</code>
<code>'12' + '3'</code>	<code>'123'</code>
<code>12 + 3</code>	<code>15</code>

O operador ***** repete uma string um número inteiro de vezes.

Exemplo de concatenação

```
mensagem = 'Bom'
mensagem += ' dia'
print(mensagem)
```

Saída:

Bom dia

Para juntar texto e números, converter:

```
idade = 20
texto = 'Idade: ' + str(idade)
print(texto)
```

Strings são imutáveis

❌ Tentativa inválida

```
texto = 'Gato'
texto[0] = 'R'
```



TypeError:

```
'str' object does not  
support item assignment
```

✅ Solução correta

```
texto = 'Gato'
novo = 'R' + texto[1:]
```



texto

'Gato'

novo

'Rato'

Uma operação
pode produzir outro
valor de texto,
sem alterar os
caracteres da string
original.

Tipo	Elementos	Mutabilidade
str	caracteres	imutável
list	valores de vários tipos	mutável
tuple	valores de vários tipos	imutável



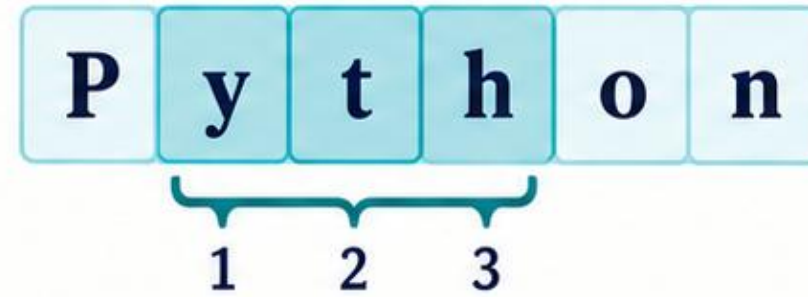
Reatribuir uma variável não modifica a string anterior.

Slicing: extrair uma parte

Sintaxe:

```
texto[início:fim:passo]
```

Índices: 0 1 2 3 4 5



```
texto = 'Python'  
texto[1:4] => 'yth'
```

- 1 O índice de **início** é incluído na extração.
- 2 O índice de **fim** é excluído da extração.
- 3 O **passo** indica de quantos em quantos caracteres se avança e, por omissão, é 1.

Expressão	Resultado	Significado
<code>texto[:3]</code>	<code>'Pyt'</code>	Do início até ao índice 3 (excluído).
<code>texto[3:]</code>	<code>'hon'</code>	Do índice 3 até ao fim.
<code>texto[-3:]</code>	<code>'hon'</code>	Os últimos 3 caracteres.
<code>texto[:]</code>	<code>'Python'</code>	Todo o texto.

A extração não altera o texto original.

Passos, inversão e limites

Uma string pode ser percorrida com passos e limites utilizando fatias.

```
texto = 'ABCDEFGHIJ'
```

0	1	2	3	4	5	6	7	8	9
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
A	B	C	D	E	F	G	H	I	J

Expressão	Resultado	Descrição
texto[::2]	'ACEGI'	Caracteres de dois em dois (a partir do início).
texto[1:8:2]	'BDFH'	Do índice 1 ao 7, de dois em dois.
texto[::-1]	'JIHGFEDCBA'	Inversão da string.
texto[20:]	''	Limite inicial além do tamanho resulta numa string vazia.
texto[:20]	'ABCDEFGHIJ'	Limite final além do tamanho não causa erro.

- 1 O passo não pode ser zero.
- 2 Um passo negativo percorre no sentido inverso.
- 3 O acesso com um índice fora dos limites, como texto[20], provoca um IndexError, mas as fatias ajustam automaticamente limites excessivos.

Verificação de palíndromo

```
palavra = 'radar'  
print(palavra == palavra[::-1])
```

Resultado:

True

Extraír campos de um código

O código **2026-LEI-042** tem comprimento 12 e é composto por três partes separadas por hífen, nos índices 4 e 8.

```
codigo = '2026-LEI-042'  
ano = codigo[:4]  
curso = codigo[5:8]  
numero = codigo[9:12]
```



ano	curso	número
2026	LEI	042

Validar a estrutura do código

```
len(codigo) == 12 and codigo[4] == '-' and codigo[8] == '-'
```

Verificar o comprimento antes de aceder aos índices 4 e 8.
O **and** avalia da esquerda para a direita e interrompe quando encontra **False**.

Esta condição verifica apenas a **estrutura** do código, **não o conteúdo** dos campos.

Depois validar o conteúdo dos campos.

PESQUISA DE SUBSTRINGS

A pesquisa de substrings permite verificar se uma cadeia de caracteres aparece noutra e obter informações sobre a sua posição e número de ocorrências.

Expressão	Resultado
'Python' in texto	True
'python' in texto	False
'Java' not in texto	True
texto.find('útil')	9
texto.find('Java')	-1
'banana'.count('a')	3

```
texto = 'Python é útil'
```

O operador **in** testa a presença de uma substring numa cadeia de caracteres.
A pesquisa distingue maiúsculas de minúsculas.

O método **find** devolve a primeira posição da substring ou -1 se não for encontrada.

O método **count** conta o número de ocorrências não sobrepostas da substring.

Índices de
texto = 'Python é útil':

P	y	t	h	o	n		é		ú	t	i	l
0	1	2	3	4	5	6	7	8	9	10	11	12

Métodos de teste

Expressão	Resultado	Significado
<code>'Ana'.isalpha()</code>	True	só letras
<code>'2026'.isdigit()</code>	True	só dígitos
<code>'ABC123'.isalnum()</code>	True	letras ou dígitos
<code>' \t'.isspace()</code>	True	só espaços em branco
<code>'python3'.islower()</code>	True	letras em minúsculas
<code>'PYTHON3'.isupper()</code>	True	letras em maiúsculas



Todos estes testes devolvem **False** para `' '`.



isdigit não valida todos os números: sinais e separadores decimais não são dígitos.

`'-12'.isdigit()` False

`'15.5'.isdigit()` False



Para converter entradas numéricas, usar **int** ou **float** com **try-except**.

Normalizar e limpar texto

Código em Python

```
entrada = '  Lei  '  
codigo = entrada.strip().upper()  
print(codigo)
```



Saída

LEI

O método **strip** remove espaços em branco nas extremidades, sem eliminar espaços interiores.

Os métodos devolvem novas *strings*.

Legenda:

- = espaço em branco (não é texto)

Expressão	Resultado (para texto = ' Ana Silva ')
texto.strip()	'Ana•Silva'
texto.lstrip()	'Ana•Silva••'
texto.rstrip()	'••Ana•Silva'
texto.lower()	'••ana•silva••'
texto.upper()	'••ANA•SILVA••'

Prefixos, sufixos e substituição

Prefixos e sufixos

Os métodos `startswith()` e `endswith()` permitem verificar se uma cadeia de caracteres começa ou termina com um dado padrão.

```
ficheiro = 'relatorio_2026.txt'
ficheiro.startswith('relatorio_')
ficheiro.endswith('.txt')
```

Prefixo
'relatorio_'

relatorio_2026.txt

Sufixo
' .txt'

```
ficheiro.startswith('relatorio_') → True
ficheiro.endswith('.txt') → True
```

Substituição de substrings

O método `replace()` permite substituir ocorrências de uma substring por outra, devolvendo uma nova cadeia de caracteres.

```
texto = 'Python é difícil'
novo = texto.replace('difícil',
                    'interessante')
```

Original

'Python é difícil'

replace()

'Python é interessante'

A substituição devolve uma nova cadeia de caracteres. A cadeia original mantém-se inalterada.

Número de ocorrências

O método `replace()` substitui todas as ocorrências por omissão. Pode ser indicado o número de substituições a realizar.

```
'banana'.replace('a', 'o', 1)
```

'banana'



'bonana'

O terceiro argumento indica o número máximo de substituições. Neste exemplo, apenas a primeira ocorrência de 'a' é substituída.

O método **replace** substitui **todas** as ocorrências por omissão.

Uma função de validação

Regra:

Identificador com 5 a 10 caracteres, apenas letras e algarismos, primeiro carácter letra.

Identificador	Resultado
'Ana12'	True
'12Ana'	False
'Ana-1'	False
'Ana'	False

Código Python:

```
def identificador_valido(texto):  
    texto = texto.strip()  
    if not 5 <= len(texto) <= 10:  
        return False  
    if not texto.isalnum():  
        return False  
    return texto[0].isalpha()
```



O comprimento é validado antes de aceder a texto[0].



Estas regras pertencem à aplicação; não são as regras gerais dos identificadores Python.

Dividir texto com split

```
frase = 'Python é muito útil'
```

```
>>> frase = 'Python é muito útil'
>>> palavras = frase.split()
>>> palavras
['Python', 'é', 'muito', 'útil']
>>> len(palavras)
4
>>> palavras[0]
'Python'
>>> palavras[-1]
'útil'
```

```
'Ana Silva'.split()
['Ana', 'Silva']
```

Sem argumento, **split** separa por espaços em branco e agrupa separadores consecutivos.

'Python é muito útil'



Python

é

muito

útil

lista de strings

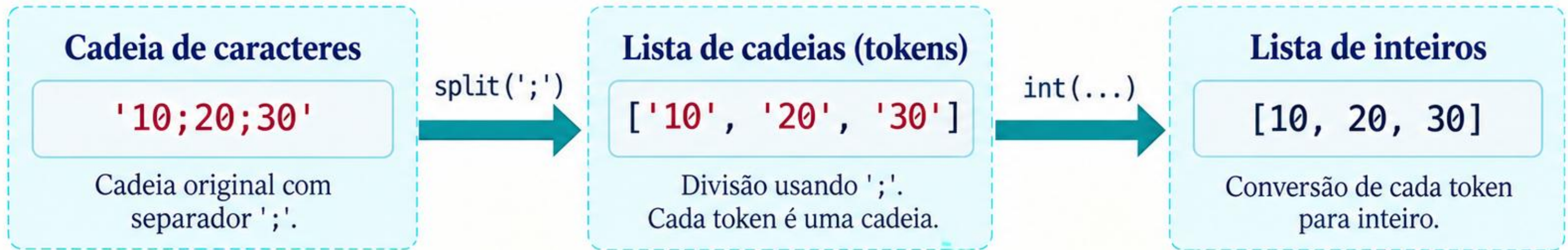
O resultado é uma lista de strings.

Separadores e conversão de tokens

Código em Python:

```
dados = '10;20;30'  
partes = dados.split(';')  
numeros = [int(parte) for parte in partes]
```

Tokens: são as partes obtidas pela divisão.



Um separador explícito preserva campos vazios.

Exemplo: `'10;;30'.split(';') → ['10', '', '30']`

`int('')` causa `ValueError`.

Definir uma constante evita repetir o separador:

```
SEPARADOR = ";"
```

Converter uma linha num registo

Linha de entrada

```
linha = '101;Ana;16.5'
```

A linha contém três campos separados por ponto e vírgula.

Código em Python

```
campos = linha.split(';')  
if len(campos) == 3:  
    aluno = (  
        int(campos[0]),  
        campos[1].strip(),  
        float(campos[2])  
    )
```

Registo resultante

```
(101, 'Ana', 16.5)
```

↑
int

↑
str

↑
float



Validar o número de campos antes de aceder às posições.



Este formato simples pressupõe que o separador não aparece nos campos. Para CSV com aspas e separadores internos, usar o módulo **csv**.

Validar e interpretar um registo

```
def criar_aluno(linha):  
    campos = linha.rstrip('\n').split(';')  
    if len(campos) != 3:  
        return ()  
    try:  
        numero = int(campos[0])  
        nome = campos[1].strip()  
        nota = float(campos[2])  
    except ValueError:  
        return ()  
    if not nome or not 0 <= nota <= 20:  
        return ()  
    return (numero, nome, nota)
```

Exemplo de registo válido

101;Ana;16.5

Linha de entrada



(101, 'Ana', 16.5)

Resultado da função

Exemplo de registo inválido

102;Rui;abc

Linha de entrada



()

Resultado da função

Convenção desta função: o tuplo vazio indica um registo inválido.

Processar um ficheiro de registos

O programa lê um ficheiro de texto, processa cada linha com a função **criar_aluno**, guarda os alunos válidos e conta os registos inválidos.

Ficheiro de entrada: alunos.txt

```
101;Ana;16.5
102;Rui;abc
103;Carla;8.0
```

Cada linha representa um registo de aluno (id;nome;nota).

A instrução **continue** passa à linha seguinte, ignorando o resto das instruções do ciclo.

Código Python:

```
alunos = []
invalidos = 0
try:
    with open('alunos.txt', encoding='utf-8') as f:
        for linha in f:
            aluno = criar_aluno(linha)
            if not aluno:
                invalidos += 1
                continue
            alunos.append(aluno)
except OSError as erro:
    print('Falha de acesso:', erro)
else:
    print('Válidos:', len(alunos))
    print('Inválidos:', invalidos)
```

Resultado da execução:

```
Válidos: 2
Inválidos: 1
```

Processamento



alunos.txt

Ler cada linha

criar_aluno(linha)

Se o registo for inválido
incrementar **invalidos**
e continuar.

Caso contrário,
adicionar o aluno à lista
alunos.

Pesquisar nomes e contar palavras

1. Pesquisar nomes numa lista

Lê um termo, normaliza-o e mostra os alunos cujo nome contém esse termo.

```
termo = 'ANA'
termo = termo.strip().lower()
for numero, nome, nota in alunos:
    if termo in nome.lower():
        print(numero, nome)
```

Dados (lista de alunos)

```
alunos = [
    (101, 'Ana Silva', 16.5),
    (102, 'Bruno', 13.0)
]
```



Saída

101 Ana Silva

Observação: o termo é convertido para minúsculas e são ignorados espaços em branco no início e no fim.

2. Contar palavras num ficheiro

Lê um ficheiro de texto, conta o número total de palavras e mostra o resultado.

```
total = 0
with open('texto.txt', encoding='utf-8') as f:
    for linha in f:
        total += len(linha.split())
print(total)
```

Ficheiro texto.txt

Python é útil
Vamos praticar



Saída

5

Observação: Aqui, palavra significa um token separado por espaços em branco.

Códigos aleatórios e gráficos

1. Gerar um código aleatório

O módulo **random** permite gerar caracteres escolhidos aleatoriamente de uma sequência especificada.

```
import random
CARACTERES = 'ABCDEF0123456789'
codigo = ''
for _ in range(6):
    codigo += random.choice(CARACTERES)
print(codigo)
```

Escolhe 6
caracteres
aleatórios



Código gerado
(por exemplo)

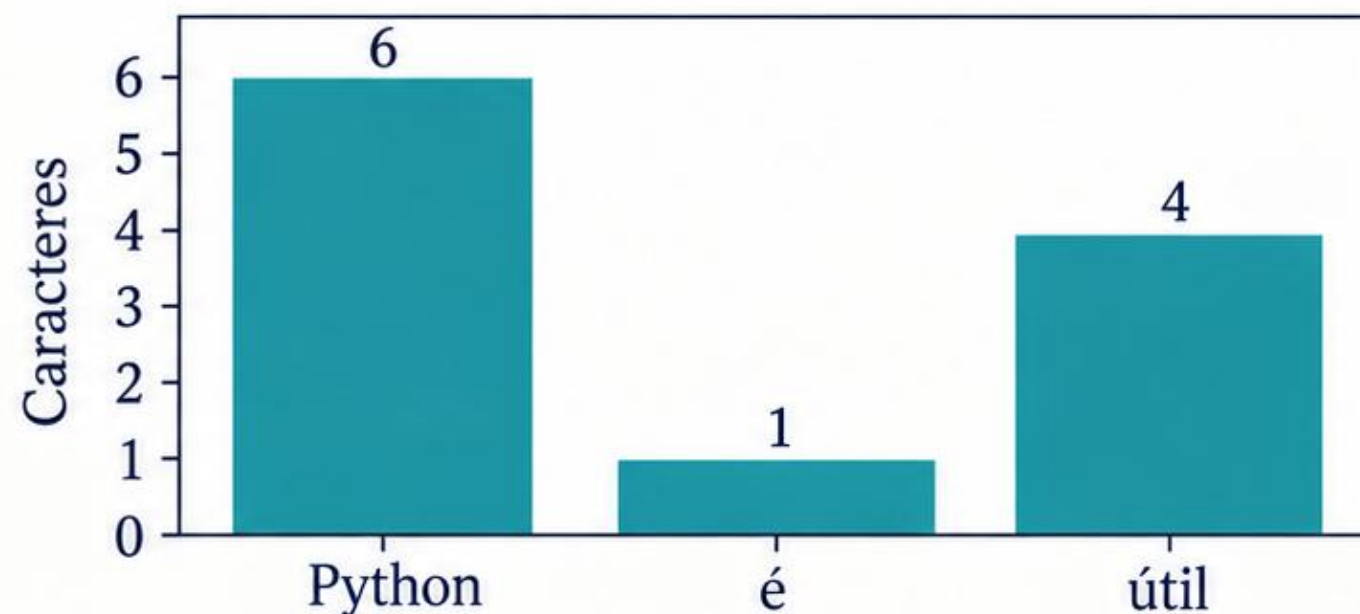
A3F09C

Exemplo didático: **random** não é adequado para senhas ou tokens de segurança.

2. Criar um gráfico de barras

A biblioteca **matplotlib** permite criar gráficos de forma simples.

```
import matplotlib.pyplot as plt
palavras = 'Python é útil'.split()
plt.bar(palavras, [len(p) for p in palavras])
plt.ylabel('Caracteres')
plt.show()
```



Exercício: processar dados de alunos

Objetivo: ler e processar dados de alunos num ficheiro de texto.

Ficheiro de entrada



```
101;Ana Silva;16.5
102;Bruno;13.0
103;Carla;8.0
104;Rui;abc
```

Um registo por linha no formato número;nome;nota.

Processamento

Ler cada linha do ficheiro

Validar com criar_aluno

Contar válidos, inválidos e calcular a média

Requisitos:

- Ler o ficheiro alunos.txt com um registo por linha no formato número;nome;nota.
- Validar cada linha com criar_aluno.
- Contar o número de registos válidos e inválidos.
- Pesquisar nomes sem distinguir maiúsculas e minúsculas.
- Calcular a média das notas apenas se existir pelo menos um aluno válido.
- Reutilizar as funções e os ciclos apresentados nesta aula.

Resultados esperados

Registos válidos: 3
Registos inválidos: 1
Média das notas: 12.5

Pesquisa de nomes

Termo de pesquisa: 'ANA'
Resultado: encontra o aluno 101 Ana Silva.

Testar: incluir ficheiro vazio, campos em falta, nomes vazios e nota fora de intervalo.

Solução: pesquisa e média

Código Python

```
print('Válidos:', len(alunos))
print('Inválidos:', invalidos)
if alunos:
    notas = [aluno[2] for aluno in alunos]
    print('Média:', sum(notas) / len(notas))
else:
    print('Sem alunos válidos.')

termo = input('Nome a procurar: ').strip().lower()
if termo:
    for numero, nome, nota in alunos:
        if termo in nome.lower():
            print(numero, nome, nota)
else:
    print('Introduza um termo de pesquisa.')
```

Exemplo de execução

Entrada do utilizador

Nome a procurar: ANA

Saída do programa

Válidos: 3
Inválidos: 1
Média: 12.5
101 Ana Silva 16.5

O que faz o programa?

- 1 Mostra o número de registos válidos e inválidos.
- 2 Calcula e apresenta a média das notas dos alunos válidos.
- 3 Lê um termo de pesquisa e mostra os alunos cujo nome contém esse termo (ignorando maiúsculas/minúsculas).
- 4 Se não for introduzido um termo, apresenta uma mensagem.



Conceitos principais

- Strings: extrair, testar e transformar.
- Split: separar campos.
- Tuplos e listas: organizar registos.